Simulating Special Relativity in a Virtual Reality Environment

6CCP3131 Third Year Project in Physics

Contributors

El Ducluzeau (K22048402)
Yo Watanabe
Leyan Ouyang
Prithyve Maheswaran
Alex Teevan
Sam Yi Wong
Thomas Koshy Thomas

Supervisor: Prof. Eugene Lim

King's College London
Department of Physics
April 17, 2025

Abstract

This report deals with the idea of using virtual reality technologies to simulate travel at relativistic velocity, the process of developing of such a software, and an evaluation of the software once created. Through use of the Unity game engine, the 3D modelling software Blender, and the Oculus Quest 2 virtual reality headset, we allowed a user to experience moving down a city street while experiencing special relativistic phenomena such as relativistic aberration, Doppler shifting, the spotlight effect, and Terrell rotation. We then compared our simulation against known results and existing simulations, finding it to be accurate to reality until near-light speed, where computational constraints limit the model and cause unphysical effects.

Contents

1. Introduction
2. Theoretical Background
    2.1. Lorentz Transformations
    2.2. Relativistic Aberration
    2.3. Doppler Shifting
3. Computational Implementation
    3.1. Setup and Criteria
    3.2. Implementing Relativistic Aberration
        3.2.1. Length Contraction, Modular Velocity, & Camera Movement
        3.2.2. Length Contraction, Modular Velocity, & Camera Movement Challenges
        3.2.3. Validating Length Contraction
        3.2.4. Relativistic Aberration
        3.2.5. Relativistic Aberration Challenges
        3.2.6. Validating Relativistic Aberration
    3.3. Doppler Shift Implementation
        3.3.1. Shaders
        3.3.2. Doppler Shift Challenges
        3.3.3. Validating Doppler Shift
4. Virtual Reality Support
    4.1. Configuring the Target Device
    4.2. Implementing VR In Unity

5. Results
6. Discussion
7. Conclusion

1 Introduction

In 1905, Albert Einstein published his paper *On the Electrodynamics of Moving Bodies*, detailing how the laws of physics change as an object's velocity approaches the speed of light. Named *special relativity*, his theory predicts complicated outcomes that most consider difficult to explain or understand, such as the shortening in length of an object's surroundings the faster it travels, or the change in colours that said object would perceive. Through the use of virtual reality, we can visualise these effects and allow a user to experience them first-hand, providing an alternative way to learn about how special relativity works.

The project's goal is to accurately simulate relativistic travel down a city street from a first person perspective, including as many relativistic effects as we can. We split off into two teams, one tasked with figuring out the theories we had to implement and making the environment in Blender, and one tasked with computationally implementing the theories described by the other team. The theoretical team had to gather equations to describe aberration angle and its consequences, which changes the apparent position, length, and curving of surroundings perceived by an observer, as well as Doppler shifting, the apparent change in wavelength of light reaching the observer which may make objects appear redder or bluer than usual. They described their findings to the computational team, who used Unity and the Oculus Quest 2 to create a software illustrating these effects according to the goal. Aberration angle was simulated by making the virtual environment out of *mesh* objects, 3D objects created through mathematically defining a network of vertices and triangles, then changing the position of the meshes' vertices in real time. Doppler shifting was simulated by using a program known as a *shader*, which changes the entire simulation's lighting and visual effects as needed. Throughout development, both teams had to work together to overcome various challenges and construct an optimal simulation, and would regularly validate their procedures and results.

This report focuses primarily on the initial implementation of length contraction, the conversion of the software to a VR software, and the challenges faced by the computational team in translating theory to software that runs on a host computer and a guest VR headset.

2 Theoretical Background

The closer to the speed of light an observer travels, the greater the special relativistic phenomena they experience, scaling cascadingly with their velocity relative to the speed of light. The two primary effects experienced are relativistic aberration and relativistic Doppler shifting, which consequentially give rise to Terrell rotation and the spotlight effect. For this project, we consider an observer travelling at relativistic speeds along a straight

line. The observer would perceive themselves as standing still while their environment moves towards them at relativistic speeds; we denote this frame of reference as the primed frame. The environment would perceive itself as standing still while the observer travels down it at relativistic speeds; we denote this frame of reference as the unprimed frame.

2.1 Lorentz Transformations

In the theory of special relativity, it is treated as a fundamental property that light always moves at the speed of light c no matter how it is observed. Consequently, an observer travelling at a velocity close to c ("relativistic speeds") would perceive the passage of time around them to slow down and their surroundings as physically shortening in length. These effects are known as time dilation and length contraction, respectively. To illustrate why these effects occur at relativistic speeds, consider a train of length L and height h travelling at velocity v. Also consider two observers, one on the train which we denote as the primed frame (whose variables are represented with a '), and one on a stationary platform observing the speeding train which we denote as the unprimed frame.

Inside the train, a device emits a light pulse upwards, which reflects off a mirror and returns downwards. From the perspective of the observer on the train, the light travels up then back down again without moving horizontally at all, having travelled a total distance of $2h$ in a time of $\frac{2h}{c}$, which we denote as proper time t'. From the perspective of the observer on the platform, the train moves horizontally by a distance vt over time t as the light travels a vertical distance of $2h$. Since light always travels at c no matter what, the only way this can make sense is if this diagonal path observed from the platform takes longer for the light to complete than the strictly vertical one as observed aboard the train. This difference in time taken between frames is time dilation; time measured in the stationary unprimed frame is dilated relative to time measured in the moving primed frame.

Now, we return to the train. Consider a light pulse travelling horizontally from the back of the train, which reflects off a mirror at the front and returns to the back. From the perspective of the observer on the train, the light pulse takes some time $\frac{2L}{c}$ to complete its journey. From this, the observer on the train can measure the length of the train, which is denoted as the proper length L'. From the perspective of the observer on the platform, the light pulse is still moving at a velocity c, but the train is moving at a velocity v, which means that the light pulse will no longer take a time $\frac{2L}{c}$ to complete its journey; the length measured by the observer on the platform will be different. This difference in lengths of an object between frames is length contraction; length measured in the stationary unprimed frame is contracted relative to length measured in the moving primed frame.[1]

Lorentz transformations are a mathematical model developed to describe these effects. They center around the relative velocity variable β, which is defined as v/c for an object moving at a velocity v. From this, we can define the Lorentz factor γ and a set of equations to translate between positions and times in in primed and unprimed frames:

Lorentz factor

$$\gamma = \frac{1}{\sqrt{1-\beta^2}} \tag{1a}$$

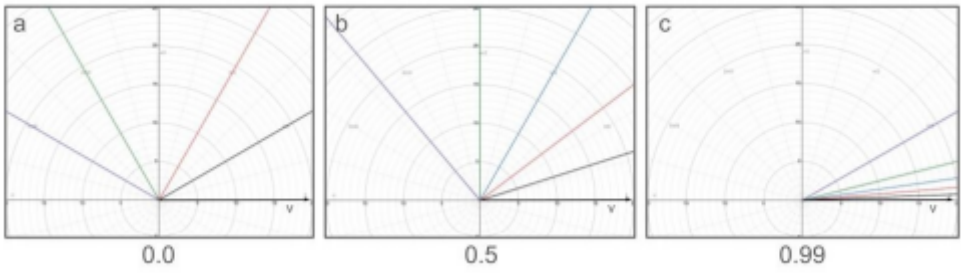


Figure 1: relativistic aberration angle θ' measured from the x-axis for β of a) 0. b) 0.5. c) 0.99. θ is the angle in the stationary frame

Coordinate transformations

$$x' = \gamma(x - vt) \tag{1b}$$

$$y' = y \tag{1c}$$

$$z' = z \tag{1d}$$

Time dilation

$$t' = \gamma(t - \frac{vx}{c^2}) \tag{1e}$$

Length contraction

$$L' = \gamma L \tag{1f}$$

2.2 Relativistic Aberration

The effects of special relativity mean that at particularly high speeds, the time taken for light to reach the observer is no longer trivial. This causes significant shifts in the perceived position and shape of an observer's surroundings, which causes the *spotlight effect*, the compression of the observer's field of view towards the direction of motion as their velocity approaches c, as demonstrated in Figure 1. This compression can be mathematically described via an angle denoted the *aberration angle* θ', which is the angle between the direction of motion and the object being distorted as described in the primed frame.[2] It is given by the following formula:

$$\cos\theta' = \frac{\cos\theta + \beta}{1 + \beta\cos\theta} \tag{2}$$

where β remains the observer's relative velocity and θ is, in the unprimed frame, the angle between the direction of motion and the object being distorted. This also causes curvature of objects offset from the line of travel.

This aberration also causes Terrell rotation, which is the perceived rotation of objects when observed by a relativistic observer as opposed to a stationary one. Consider a cube whose faces are aligned with the *xyz* axes. A stationary observer not undergoing relativistic effects would observe at most three faces of the cube. However, due to the time taken for the light emitted by the cube to reach an observer's eye, a relativistic observer may observe faces that would normally be hidden from view at that position, causing the cube to appear rotated.

2.3 Doppler Shifting

Doppler shifting is observed in everyday life with sound. When a sound emitter approaches an observer, the time that elapses between sound waves reaching the observer decreases as the source approaches, then increases again as the source recedes. This causes a perceived shift in wavelength, which is experienced as for example the changing pitch of a passing car's engine. At relativistic speeds, this also happens with light, wherein the apparent change of wavelength changes what colour an observer will perceive their surroundings to be.[3] The relativistic Doppler shift can be described by the following equation:

$$\lambda' = \lambda\gamma'(1 - \beta\cos\theta) \tag{3}$$

where β remains the relative velocity, λ is the wavelength emitted in the unprimed frame, θ is the angle between the observer's velocity vector and the direction to the source in the stationary frame, and λ' is the wavelength perceived in the primed frame. Relativistic Doppler shifting is at its most apparent when the observer's velocity vector and the source direction are parallel in the stationary frame. If the observer faces forward, light comes from in front of them and $\cos\theta$ = -1, leading to:

$$\lambda' = \lambda\frac{1-\beta}{\sqrt{1-\beta^2}}$$

Here, $\lambda' < \lambda'$; the observed wavelength appears shorter, which makes the surroundings appear bluer. This is known as blueshift. If the observer faces backwards, light comes from behind them and $\cos\theta$ = 1, leading to:

$$\lambda' = \lambda\sqrt{\frac{1+\beta}{1-\beta}}$$

Here, $\lambda' > \lambda'$; the observed wavelength appears longer, which makes the surroundings appear redder. This is known as redshift. If the observer faces sideways, light comes in perpendicular to them and $\cos\theta = 0$, reducing the equation to $\lambda' = \lambda\gamma$; there is still relativistic Doppler shifting. For other values of θ, the exact nature of the shift varies non-linearly.

3 Computational Implementation

With the theory behind special relativity as perceived by an observer travelling along a straight line in place, the computational team had to translate these theories into a piece of VR software that can be experienced by an end user. To achieve this, the computational team worked separately on different parts of the project before combining them into one final product. This necessitated the use of heavy coordination, project standards, and a list of criteria which if achieved meant we could consider the project successful.

3.1 Setup and Criteria

The project is a software that allows an end user to equip a VR headset and experience relativistic motion down a city street, complete with all of the aforementioned effects. The following software and tools were used to build it:

- Unity Hub & Unity Editor 6000.0.34f1: hosting and displaying the objects and programs making up the software in a virtual space known as a *scene*, development of code
- Universal Render Pipeline (URP): shader effects
- InputSystem, OpenXR Plugin, XR Interaction Toolkit: VR support
- Dell Latitude 5540: host computer running the software. Windows 10 OS. 14 cores, 20 threads, 64GB of RAM
- Oculus Quest 2: VR headset. 72Hz refresh rate, 1832x1920 per-eye resolution
- Blender 3.4: low-poly modelling of London landmarks which are then imported into the Unity editor

Before beginning programming, the computational team had to decide on a set of standards for everyone to follow, to ensure that the three parts of the project would work when put together. First, the list of relativistic effects that had to be simulated in order to consider the project a success were identified:

- Increasing curvature of straight objects as β increases
- Compression of vision in the motional direction as β increases
- Spotlight effect where $\beta > 0.95$

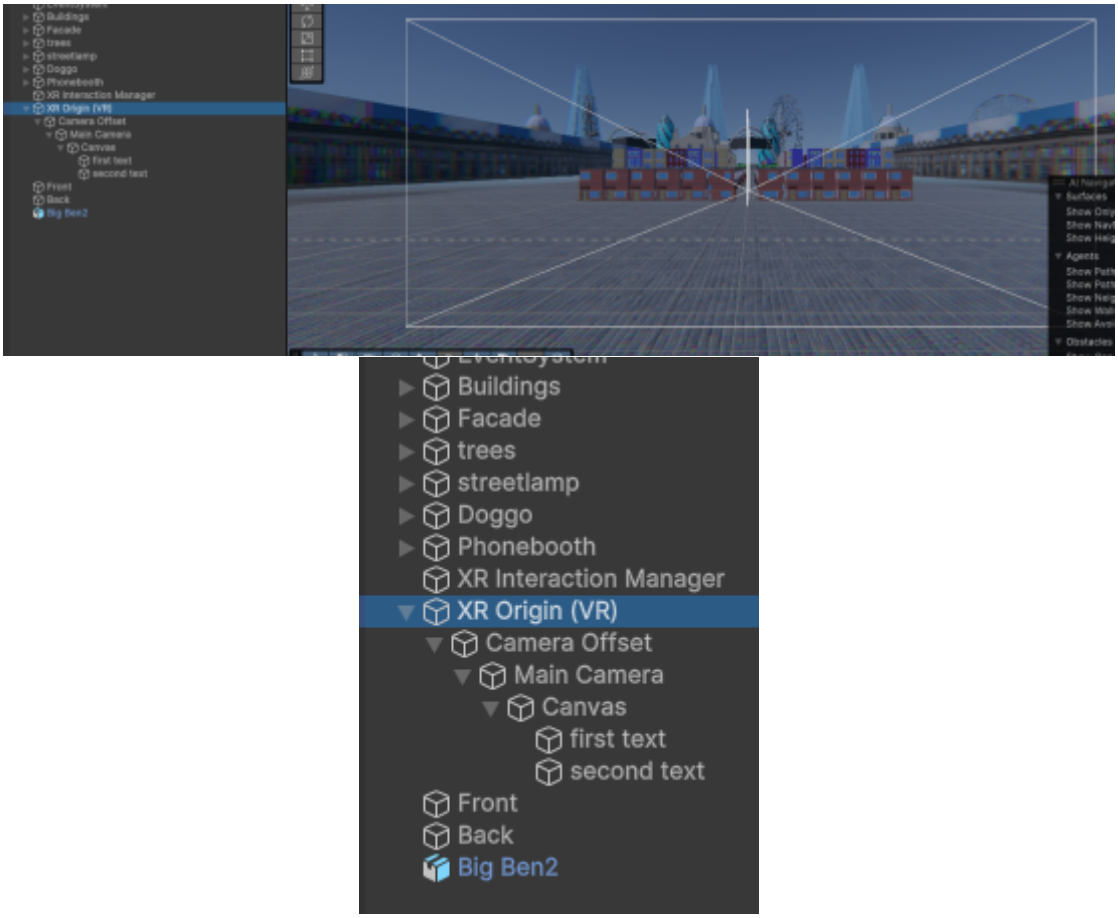


Figure 2: MainCamera (edited to support VR) GameObject and its in-scene field of view in the Unity Editor. Below is a legible capture of the MainCamera and its children of hierarchy.

- Increasing Terrell rotation as β
- Blueshift when facing forward
- Redshift when facing backwards

The team members responsible for programming relativistic aberration trialled a solution using shaders, programs that alter the visuals of a scene in a video game, or meshes, a protocol that creates 3D objects in a video game through mathematically defining the objects' vertices; the mesh solution was deemed easier to implement. They then trialled a solution where the program would move a player object while the surrounding environment remained still, but a solution where the player is still and the environment moves was deemed easier to implement. For Doppler shifting, a combination of shaders was deemed the most efficient solution. The environment was constructed out of 3D models and textures imported from Blender, which would be converted to meshes automatically by Unity's built-in features. Thus we get the following constraints:

- Observer is fixed at the origin (0,0,0)

- We treated this as the primed frame. In theory, this should be moving, but it was programmed this way for ease of implementation
- The street is moved continuously along the x-axis
- We treated this as the unprimed frame. In theory, this shouldn't be moving, but it was programmed this way for ease of implementation
- Axis conversion between Blender and Unity was necessary
- Blender uses z as the vertical direction, while Unity uses y

3.2 Implementing Relativistic Aberration

3.2.1 Length Contraction, Modular Velocity, & Camera Movement

The relativistic aberration team chose to break down the problem of implementing such an effect into constituent parts. Upon closer inspection with assistance from the theoretical team, it was found that the formulae for relativistic aberration necessarily included length contraction. Thus, it was chosen to implement length contraction first and sort out relevant challenges, then upgrade the program to relativistic aberration. Two solutions were considered for relativistic aberration. One was a combination of shaders to distort the visuals in such a way that matched the predicted outcome from relativistic aberration. The other was to use meshes and directly manipulate the positions of their vertices to simulate relativistic distortion.[4] We figured out an implementation which uses meshes very quickly, so we chose to use that. In order to provide the end user with a deeper understanding of the connection between an observer's velocity and special relativistic effects, it was elected to allow the observer to change their relative velocity in real time, as the software ran. The problem of length contraction could then be further decomposed into sub-problems:

- Implementing and automating mesh object creation
- Coding an algorithm to update the mesh objects' vertices' positions every frame the software runs
- Implementing length contraction
- Allowing the user to move the camera
- Allowing the user to change their relative velocity and camera sensitivity
- Keeping track of the user object's total displacement
- Calculating the correct mesh position each frame as a function of total displacement and length contraction

These were solved in the order listed. We first placed a [MainCamera] game object in the scene, which is an object that allows the program to visually display the scene, and assigned the coordinates (0,0,0). From there, testing was done on three 2D square objects. Internally, meshes are an array of [Vector3] objects, which themselves are Unity programming structures representing 3D vectors. Each [Vector3] in the array represents the coordinate positions of one of the mesh object's vertices. For each square, we defined four [Vertex] points, which are 3D coordinate points in the software that define the location of a vertex on a shape. These vertices were then used to define two [Triangle] objects, 3D objects in the game which are triangles. These triangles themselves were finally used to define a [Mesh] object. With the meshes defined, an algorithm was then written which simply added a fixed value in the x direction to each vertex, every frame. To allow the user to change their relative velocity, we define a new variable β in the MainCamera's [void start] routine to represent the relative velocity; each program in the software has its own [void start] routine, and it is run once when the software is launched. We also added to the MainCamera a new variable γ to represent the Lorentz factor, calculated every frame according to Equation 1.1. We defined a new array of [Vector3]s for each mesh, the "current position" array, which was set each frame to the vertices' current position scaled by γ according to Equation 1.6. We added a *heads-up display*, a small graphic that conveys important information to the end user. We attached a [Canvas] object, which is an object that enables programmers to create and display a heads-up display in the scene, to the MainCamera as its child in Unity's object hierarchy. An object being a child of another object in Unity means that it inherits certain properties of its parent objects, such as position, angular, or scale transforms. Through this [Canvas], we defined a child [Slider] object, which is a graphical element that allows the user to change a value by sliding a bar horizontally. A routine was added to the [MainCamera] which detected when the user moved the slider, and increased or decreased β accordingly. We also added another routine to the [MainCamera], which detected when the user moved their mouse, saved this input as a [Vector3], and performed an angular transform on the [MainCamera] to update its facing direction to match the [Vector3]. We also added to the [Canvas] another [Slider] which allowed the user to change the camera's sensitivity through a reuse and modification of the code used for the β slider.

The last two steps required overhauling of the code up to this point. Since the observer's velocity could now change, we could no longer add a fixed value in the x direction to move the vertices like we had been doing up to now. Instead, total displacement and length contraction had to be resolved separately. The total displacement tracker was implemented first. We redefined the arrays defining the meshes as the "starting position" arrays. Then we created a new array of [Vector3] objects for each mesh, which was updated by a base value scaled by γ taken from the [MainCamera] in the x direction every frame. We created another array of [Vector3]s for each mesh, which would be used to determine the current position that the meshes should be at on the current frame. Finally, we wrote an algorithm that ran every frame, updating the displacement tracker, adding that value to the starting position arrays, then scaling that by γ according to Equation 1f, and feeding this result to the objects' current position arrays, moving and contracting them as needed.

3.2.2 Length Contraction, Modular Velocity, & Camera Movement Challenges

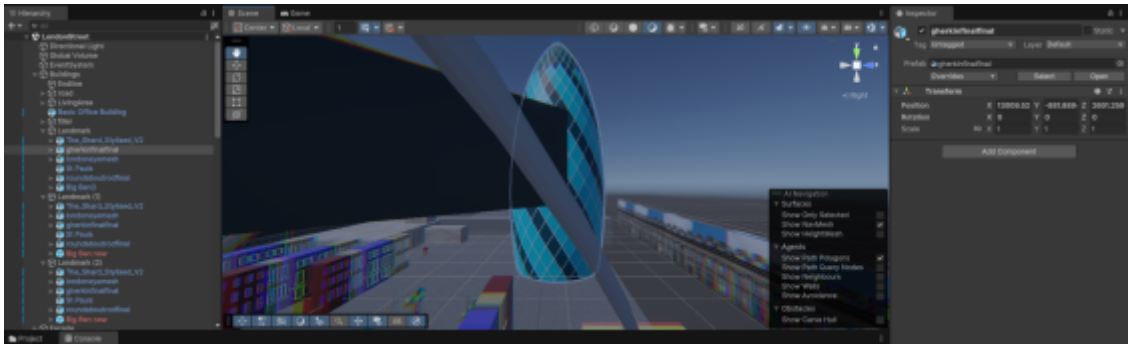


Figure 3: a mesh object of the Gherkin in the Unity scene. Though it may not be apparent at first glance, this Gherkin is actually made up of many small triangles arranged and coloured to look like the real landmark.

Implementing this solution was not as straightforward as initially assumed. First, the question of how to limit β had to be solved. It would be unphysical to allow the user to reach a β of 1, since massive objects can never travel at c, and we assumed the [MainCamera] acted as a massive object. The code was also written assuming constant motion, so we were unable to allow the user to reach a β of 0. Finally, since c is equal to 300,000,000 metres per second, attempting to recreate these speeds and distances 1 for 1 in the project would result in the scene using an impractically large scale. To solve all of these at once, we defined for simplicity's sake c as 1 Unity unit per frame and made it so that the user is unable to go below $\beta = 0.01$ or $\beta = 0.99$.

Unity deals with [Vector3] operations in a way that was inconvenient to the team. When a [Vector3] is called in the code in an addition or subtraction operation, the programmer is not calling the [Vector3] object, but rather a new temporary object with identical properties to the called [Vector3]. This means that any changes made through this method will be gone when the program deletes the new temporary object and will not actually apply to the called object at all. In order to get around this, the programmer must define a variable with a value equal to the [Vector3] they wish to edit, then directly edit each of its components individually, then finally reassign this variable as the [Vector3]'s value. To work with this limitation, we ended up in the end with a starting position array, a displacement tracker array, a current position array, and a variable for modifying the displacement tracker array. The code would individually get every component of the starting position array and assign them to the variable, then add each individual component of the displacement tracker to the corresponding component of the variable, which was finally assigned to the current position array before being scaled. Fortunately, scaling operations do not have this limitation and we did not have to make any extra arrays for it.

By this point, the end user did not have a way of knowing what their current value of β or camera sensitivity was, and it was also found to be inconvenient to have to use the slider to change β or the camera sensitivity, given that this would also move the camera. We elected to remove the sliders, replacing them with detecting left/right arrow key inputs for camera sensitivity and up/down arrow key inputs for β. We additionally added a pair of [Text] objects to the Canvas, which retrieved the values of β and camera sensitivity from the [MainCamera], then updated an on-screen graphic displaying the user's current β, γ calculated from β, and camera sensitivity to match.

```
private Mesh mesh; // The mesh of the object this script is attached to
private Vector3[] originalVertices; // Stores the original vertices of the mesh
private Vector3[] modifiedVertices; // Stores the modified vertices after calculations
private Vector3[] baseVertices; // Stores the vertices after applying scale and rotation
public float relativeSpeed = 0.5f; // Speed relative to light
private float beta1; // A variable to store the relative speed from beta controller
private GameObject objectBeta; // Reference to the GameObject with the relative
private float x_0; // Initial x position of the object
private float y_0; // Initial y position of the object
private float z_0; // Initial z position of the object
```

Figure 4: a snippet of code showing the assignment of variables, vector arrays, and other Unity components used to create and manipulate mesh objects.

3.2.3 Validating Length Contraction

In order to validate length contraction, we had to ensure that it scaled cascadingly with β. To ensure quantitative and qualitative validation, the visual output of the scene was inspected, but the value of γ for each value of β was also output in the debugging console every frame. We found that both of these criteria were met, and could proceed to the next stage of the greater problem.

3.2.4 Relativistic Aberration

With length contraction implemented, we were then able to upgrade the script to relativistic aberration. As we were moving from hand-generated meshes to meshes imported from Blender, and from the length contraction equations to the relativistic aberration equations, we had to rewrite the script from the ground up, though the method of using a network of arrays through which the displacement and scaling values are fed in a chain remained. The Blender objects had to be transformed to fit into the scene, using Unity's method of defining [Vector3] coordinates relative to the origin. We now used the three arrays of [Vector3] objects: [originalVertices] (unmodified coordinates), [baseVertices] (base coordinates scaled and rotated according to the imported mesh's parameters), and [modifiedVertices] (the final aberrated coordinates after all calculations are performed).

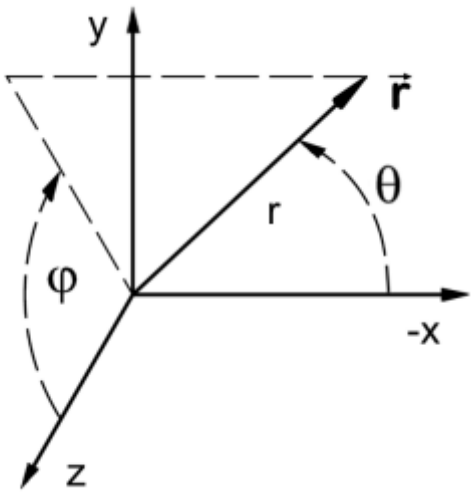


Figure 5: the 3D spherical coordinate system. r is the position vector. r is the radial distance. θ is the polar angle. ϕ is the azimuthal angle.

To apply Equation 2, we had to convert the vertex coordinates from Cartesian to spherical, aligning the polar axis with the direction of motion (-x). The polar angle θ corresponded to the angle used in Equation 2. The conversion equations are as follows:

Coordinate rescaling

$$r_i = r_0 + V_i$$

where r_i is the position of vertex i in Unity's system, r_0 is the position of the object's origin in Unity's system, and V_i is the local position of vertex i as imported from Blender

Cartesian to spherical

$$r = (x, y, z) = (-rcos\theta, rsin\theta sin\phi, rsin\theta cos\phi)$$

where r is a vertex's [Vector3] converted to spherical form, x, y, and z are its Unity-wise Cartesian coordinates, and θ and ϕ are the polar and azimuthal angles

Once we got r and θ, we performed the aberration calculation for each vertex. The azimuthal angle was invariant under Lorentz boosts, so we didn't need to aberrate it. Then, we used similar equations to convert back to Cartesian form, assigning the result to a variable we designate [rotatedVertex] which is then assigned back to [modifiedVertices] and fed back into the mesh to update its vertices' positions.

3.2.5 Relativistic Aberration Challenges

At first, our aberration looked anomalous, with the objects either moving in the wrong direction or not distorting when they were supposed to.. A review of the code found

that this was due to aligning the velocity vector with the z-axis with regards to calculating aberration rather than the x-axis. This was fixed by making sure that aberration calculations correctly used the x-axis for the motional axis.

We initially assumed that aberration only affected coordinates in one plane, so our initial program caused horizontal curvature, but not vertical curvature. Once we understood that aberration is a fully 3-dimensional effect, we updated the code to use the spherical coordinate conversion solution.

Unity's parent-child hierarchy caused some issues, where the aberration angle script's effects were one of the things that passed down from parents to children. To prevent each child aberrating an additional time per tier we went down the hierarchy and exhibiting increasingly anomalous behaviour with each aberration, we elected to only apply the aberration angle to the root parent of each hierarchical object, which would aberrate the entire hierarchy correctly.

At first, the vertex motion algorithm did not take into account the position and rotation of the aberrating object, which both affect the object's vector statistics with regards to Unity's implementation of vector maths. By not accounting for object rotation, the aberrations were wrongly distorted due to the changing rotation of the combination of moving objects and user camera rotation. To solve this, we scaled each object's [Vector3] by its rotation during the conversion step from Blender to Unity style coordinates. Once aberration is calculated, the object must be rotated by the inverse of the initial rotation in order to recover the full correct coordinates before updating its vertices.

The fully manual approach from the old length contraction script didn't scale up to potentially infinite objects, each with potentially differing amounts of vertices associated with them. In order to resolve this, we had to automate performing these operations on every vertex in the objects, by using iterative loops in the code that checked for how many vertices the object contains and repeating each step of the process once per vertex until it has applied the calculations to every vertex.

The aberration script ran on every vertex, each frame. We didn't know how many vertices we would have by the end, but we were aware that this could stack up very quickly. The host computer had to resolve every calculation before it can update the software, and if it had too many calculations to perform from too many vertices being present in the scene, it might not have been able to execute them all before the human eye could notice that the frame hadn't updated. To avoid this slowdown, we intentionally imported a massive amount of cubes that we were sure the target device couldn't handle, calculated how many vertices each cube was worth, then gradually removed cubes until we found a performance deemed acceptable for our tech demo. This vertex count was set as our *vertex budget* which was not to be exceeded by the environmental designers.

As reflection, refraction, and diffraction involve the alteration of the motion of light, they further complicate relativistic equations by a magnitude simply too large for us to simulate given the time and scope of our project. As such, we chose to fill in windows and other such surfaces in our models with a colour and texture vaguely close to that of the scene's sky to provide a passable illusion of reflection without having to accurately simulate it. This is a recognised limitation which could be alleviated with more time and work.

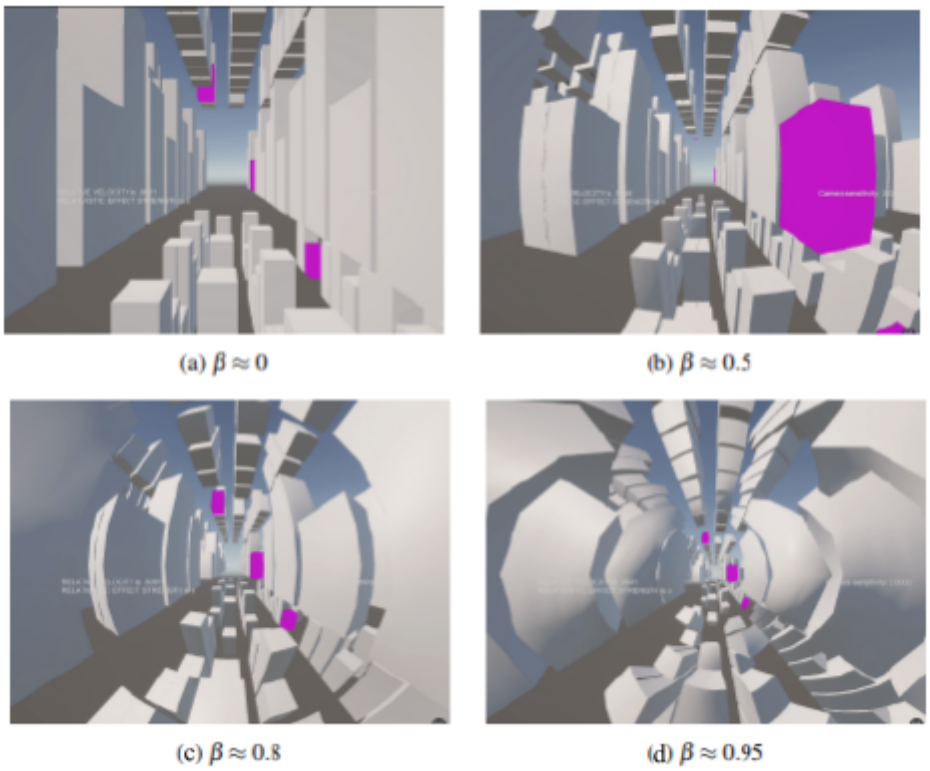


Figure 6: relativistic aberration at β of a1) 0. b1) 0.5. a2) 0.8. b2) 0.99.

3.2.6 Validating Relativistic Aberration

We validated relativistic aberration both qualitatively and quantitatively, both by observing the distortions to see if all predicted effects were present and by calculating the value of the aberration angle manually and comparing it to what the program gave us. When testing our script on our test environment, which was made of rows of cubes scattered around the observer, we were able to verify the contraction and curvature of the cubes as β increased. As β approached 0.8, Terrell rotation became apparent, and past β = 0.95, the spotlight effect began to present itself readily, as shown in Figure 6.

As β progressively increased, the percentage error between the calculated and simulated aberration angles increased too. At lower velocities, the aberration angle was accurately simulated, but past β = 0.9, the percentage error exceeded the 5% threshold, indicating inaccuracy as shown in Tables 1 and 2. We thought of possible reasons for this; one such reason could be that since vertices are moved to simulate curvature, our objects simply didn't have enough vertices to accurately simulate such extreme curvature. This indicates a limitation in the simulation that could be improved upon given further time and work.

3.3 Doppler Shift Implementation

3.2.1 Shaders

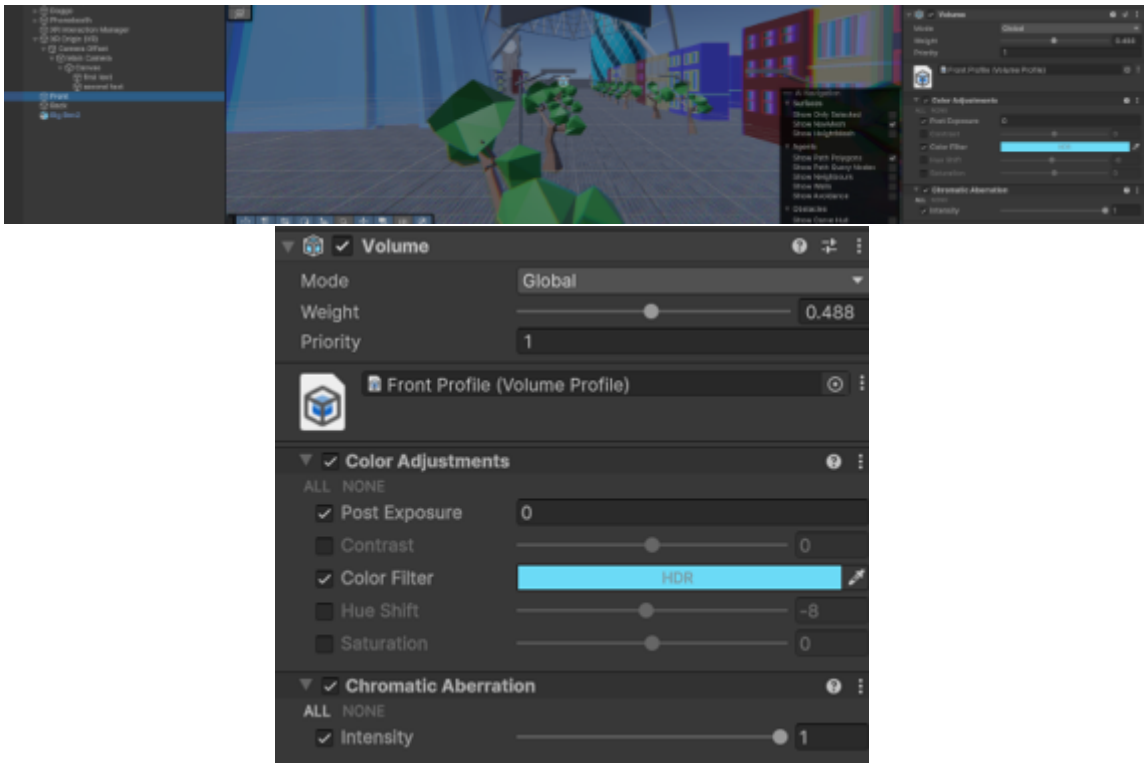


Figure 7: one of the global volumes used to track and apply colour modulation for Doppler shift simulation. Below is a legible capture of the visual modulation effects tied to this global volume.

In order to implement Doppler shifting in our software, the Doppler shift team created a custom shader applied to the scene. As relativistic Doppler shifting is sensitive to the objects' position in space, we needed a way to give the shader some positional context in the scene. To this effect, we created two [URP Global Volumes], which are invisible boundaries spanning the entire scene that can have attributes such as directions assigned to them. We applied the shaders as a post-processing effect to the global volumes, and calculated the Doppler shift using Equation Set 3, taking the user's viewing angle θ relative to the directions assigned to the two global volumes. The shader implements a smooth transition in colour shift with viewing angle, with complete blueshift at $\theta = 180$ and complete redshift where $\theta = 0$.

3.2.2 Doppler Shift Challenges

The relativistic Doppler shift scales non-linearly with non-parallel values of θ, which was too complicated for us to simulate given the time and scope of our project. Therefore, we elected to simply interpolate the red and blueshifts linearly with θ, which looked good enough for our purposes. With more time and work, this is a limitation that could eventually be surpassed.

The shaders were written early, and their implementation did not account for the rewrite of the aberration angle script to account for all three dimensions. This necessitated a slight

modification to the shader script to also account for the new surrounding algorithms before it was added to the total software.

3.2.2 Validating Doppler Shift

As we were unable to completely accurately simulate the Doppler shift for every angle, we chose to validate it qualitatively. We confirmed that blueshift was observed when looking forward and redshift was observed when looking back. Overcoming the gradual angle limitation would allow us to properly define quantitative validation of the shifting effect.

4 Virtual Reality Support

Though we had functional software that ran on a Windows computer, it was not ready for VR yet, and support for VR had to be added manually. This came in two steps. One step was to configure the host computer for VR, and the other was to modify the software to work with a VR input/output system. We chose to break this part of the project down into the two aforementioned sub-problems and solve them in that order.

4.1 Configuring the Target Device

The initial target device was a computer from King's College London, equipped with 8 cores and 16 total threads in its central processing unit, meaning that this computer could execute up to 16 instructions simultaneously per CPU cycle. It also had 32GB of RAM, volatile computer memory used to store temporary data like calculation results. Given that this software was restricted by needing to perform large amounts of calculations per frame, this configuration was ideal for our purposes. However, restrictions placed upon the computer prevented downloading software onto it so long as it used Windows as its operating system, which was a problem, as almost all modern implementations of VR only support Windows. As we could not download any software, we could not get Unity or our software as a standalone version onto the target device. To get around this, we chose to change its OS to Linux, which lacked this download restriction. From here, we needed to recover Windows on this Linux machine in order to support VR. We considered two options.

The first option was to *dual boot*, which entails installing both Windows and Linux on the same machine and giving them access to a shared part of the computer's permanent storage. The idea here was that we would download files while on Linux, put them in the shared partition, then access and run these files on Windows. However, the computer's *BIOS*, a piece of software all computers have independent of OS which allows users to configure the OS for purposes such as dual booting, was locked by a password nobody working on the project knew, which eliminated this possibility.

The other option was to run a *virtual machine*, a digitally simulated computer running on a host computer. We would run a Windows virtual machine on the Linux software, which would also be exempt from the downloading restriction, onto which we could then download and run our software. As entire simulated computers, VMs are incredibly costly on computer resources, but we were confident that this computer would be able to perform well while running our software on a VM. However, this approach also ran into some problems. Oculus Link, an app for allowing communication between a computer and a VR headset, does not work on VMs, which left our only option for VR support to be using the gaming service Steam's VR compatibility plugin developed for Unity.

We first tried creating a VM using the Oracle VirtualBox provider, however Oracle is unable to create VMs that can connect over wireless which is required to use SteamVR. We then tried to use the KVM/QEMU provider, which is able to create VMs that connect over wireless, however the procedure through which this is done was found to be poorly documented to such a degree that the option was considered impractical given the time we had available. Additionally, computers contain a component known as a graphical processing unit, which is specialised to display things such as scenes in a video game. VMs by default do not use the host computer's GPU and must be told to do so, at which point they take complete control of the selected GPU, leaving the host unable to use it. This means that to allow a VM to use a GPU, the machine must have at least two, one for the VM and one for the host. As this computer only has one GPU, the VM would have no advanced graphical simulation capabilities, which we were concerned would significantly impact its performance when running our software.

We were able to locate an Alienware M17 laptop which was used as a backup target device in case we were unable to get the KCL laptop working. This computer contained only 4 cores with 8 threads, which afforded it half the CPU power of the target machine and caused significant slowdown when running our software, making it suitable only as a last resort option. We were eventually able to procure the Latitude 5540 laptop we ended up using as the target device, which had 14 cores with 20 threads, 64GB of RAM, and could install files directly onto Windows, making it both more powerful and more convenient to use than the KCL laptop. We elected to do everything on this laptop and avoid the problem entirely. From here, we made a Steam account explicitly for the purposes of the project, installed SteamVR on the laptop, and installed Steam Link on the VR headset, then finally verified that the two could communicate.

4.2 Implementing VR In Unity

With a VR environment set up, we then had to configure our Unity project to be compatible with VR. To do this, we used the OpenXR and XR Interaction Toolkit plugins. From there, we ensured SteamVR was set as the default VR method that OpenXR would look for when the software was launched. We configured OpenXR through the provided wizard to use our Oculus Quest 2's control scheme and replaced the MainCamera object in our Unity scene with an [XROrigin] object, a user interaction object specialised for VR handling, including an automatically generated child MainCamera object with special scripts and parameters to communicate with the VR headset. Once we positioned the

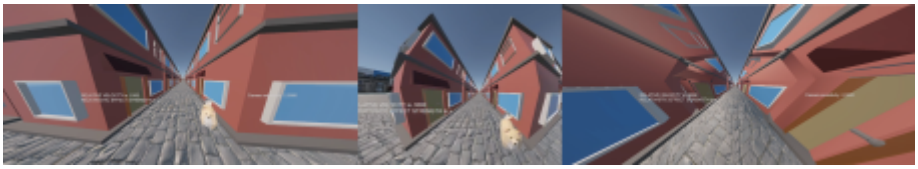


Figure 8: the simulation showing increasing distortion as β increases. Left to right: β = 0.01, 0.5, 0.9.

XROrigin at the exact coordinates and rotation formerly occupied by the MainCamera and ran the software, the plugins automatically connected it through SteamVR and Steam Link to communicate with the VR headset. This step automatically implemented head tracking as part of the plugins, allowing the end user to freely rotate the camera by moving the VR headset around. This depreciated the camera sensitivity script, which was removed. The final change to make was to remake the HUD using the [XROrigin]'s [Canvas] option and attach the global shader volumes to the [XROrigin] instead of the [MainCamera], as well as modifying the script that allows the end user to change β to detect input from the buttons on the controllers recognised by the VR headset rather than the keys on a keyboard.

5 Results

By the end of the project, we had assembled a software that allowed an end user to experience travelling straight down a street made of London landmarks in VR, with the ability to freely look around and change their relative velocity as desired in order to increase or decrease the magnitude of special relativistic effects experienced. The simulation exhibited object curvature, compression of vision, the spotlight effect, Terrell rotation, and correct Doppler shifting, making it a success according to the criteria established earlier.

However, the simulation also had its limits. The error in simulating relativistic aberration exceeded 10% beyond $\beta = 0.9$, exhibiting significant deviation frrom theory, and we were unable to quantitatively verify the accuracy of our Doppler shift implementation or implement light distortion effects such as reflection. With this in mind, we now show the simulation and how relativistic phenomena can affect a familiar-looking everyday scene.

Figure 8 displays the simulation at various relative velocities. As β increases, relativistic effects on widely recognised landmarks such as the Shard visibly increased. Occasionally at values of β deemed inaccurate under our simulation, visual inaccuracies such as objects phasing into each other occurred. Effects such as curvature and Terrell rotation were particularly prominent on the tallest landmarks such as the Gherkin or the Shard, while effects such as the spotlight effect were particularly prominent with the overhead curved object that the user passes under.

The environment designers hid six Pomeranians around the environment to encourage the end user to look around more, providing a little bit of humour and challenge to the software which could improve user engagement and experience by a noticeable amount, as well as allowing the end user to locate more landmarks to compare with everyday non-relativistic travel and experience the effects of relativity in all directions.

6 Discussion

The simulation successfully displays relativistic aberration and Doppler shifts in ramping intensity as the user increases β. The project is of significant value to the field of science communication; where some concepts are widely regarded as difficult to explain with words, VR experiences provide a more interactive visual method of teaching people advanced scientific concepts at their own pace, in a way that may be more approachable to wider audiences who may not understand a more technical literary explanation using diagrams and equations. The approach of using a street with widely recognised landmarks from London helps make the experience have an aspect of mundanity to it, which allows the end user to better connect the physics they are seeing with familiar memories and experiences from everyday life, potentially strengthening the use of such VR experiences as a teaching tool.

However, as discussed earlier, the simulation is not fully accurate. The accuracy of our relativistic aberration noticeably degrades past $\beta = 0.9$, we do not simulate light distortion effects such as reflection, and we were unable to accurately build or quantitatively verify Doppler shifting for all θ. This means that for now, technical explanations using diagrams and equations remain the best way to paint a completely accurate picture of theories such as relativity, but simulations such as ours can serve as an easily accessible entry point to introduce an audience to a concept before other teaching methods such as lectures or textbooks explain how it works behind the scenes in full detail. VR simulations could also be used as a new scientific simulation method to probe hypotheses or to better understand an existing scenario through the immersion and spatially perceptive experience offered by VR.

If we had more time and resources, we may be able to overcome all of the aforementioned limitations and build a progressively more accurate simulation, increasing the quality of the VR simulation as a teaching tool. However, we may have to rethink our approaches; an increase in vertex count may be necessary to solve certain problems such as the resolution of our aberration at particularly high values of β if we continue to use a mesh solution to aberrate. This would significantly increase performance load, so shader-based approaches may scale up better in the future instead of requiring us to acquire continuously more powerful devices; it may also make the software more accessible to those with less powerful computers.

Teamwork was constant and present from early in development, however it could have been further improved and introduced even earlier to reduce debugging time. Additional care and close inspection of the software at hand would have helped us avoid issues like coordinate system misalignment, and strengthening our peer review process could accelerate the pace of development and further improve the quality of the output.

7 Conclusion

The VR Relativity project aimed to provide an intuitive and immersive experience of the usually abstract and difficult to visualise physics of special relativistic travel through use

of the unique traits of the virtual reality environment. Through use of Unity and Blender, we built a street scene featuring a stationary observer watching the most prominent London landmarks fly by them at relativistic speeds to provide a sense of familiarity and relate the experience to everyday life, with the relative velocity being fully modular so that the user can experience as much relativity as they want. In order to develop the software, we had to implement the following:

- VR support so that a user can be immersed in the scene and control the camera by looking around
- The ability to change β using the VR headset controllers
- GameObject motion so that the street can be moved
- Relativistic aberration to distort the surroundings from the observer's frame of reference
- Post-processing effects to simulate Doppler shifting based on the observer's facing direction

Relativistic aberration was implemented by constructing the environment out of mesh objects and using C# scripts to modify the positions of their vertices, simulating contraction, curvature, and other distortive effects according to Equation 2. Quantitative analysis revealed accuracy up until $\beta = 0.9$, at which point the percentage error between the theoretical and simulated aberration angle exceeded 5%, where the simulation is considered inaccurate.

Doppler shifting was implemented using a custom shader that could shift the software's colour mappings closer to red or blue as needed, calculating colour shifts based on the viewing angle relative to the directions defined in two invisible volume objects occupying the entire scene. Qualitative validation confirmed that the scene is blueshifted when looking forward and redshifted when looking back, but we were unable to fully implement accurate Doppler shifting for all values of θ which prevented us from quantitatively validating our simulation of Doppler shift.

These inaccuracies could lead to minor but still important misconceptions about how the laws of physics work at ultra-high speeds. Additionally, the high CPU costs of the software render it inaccessible to most computers, requiring a powerful machine to run the software without significant lag. To improve the project in the future, changes could be made such as implementing relativistic aberration using shaders instead of mesh transformation to reduce CPU costs and render the project more scalable to more detailed objects or higher object counts. Another potential improvement would be to simulate a stationary environment and a moving observer, which could allow for free movement and simulation of special relativity across all three axes of motion.

Despite these inaccuracies, we still believe we have produced a unique, immersive, and valuable experience, offering an important alternative view on special relativity compared to the usual teaching with equations and stationary diagrams, which could inspire public curiosity on complicated topics through presenting easily accessible and intuitive experiences of complicated theoretical concepts such as relativistic aberration, and which demonstrates the potential for VR as a whole to become an invaluable tool in

scientific communication to the greater public due to the amount and quality of information and knowledge that can be shared through an easily approachable medium considered entertaining by the general public.

The project was incredibly interesting, offering us a deep insight into the collaborative research and software engineering experience, as well as providing a powerful lesson on the value that virtual reality could have towards scientific communication and the field of science in general if more attention were drawn towards its use in these fields.

References

1: Walker, J. (2022). C-ship: The Lorentz Contraction. *wwww.fourmilab.ch*. Retrieved April 17, 2025, from https://www.fourmilab.ch/cship/lorentz.html
2: Walker, J. (2022). C-ship: The Aberration of Light. *wwww.fourmilab.ch*. Retrieved April 17, 2025, from https://www.fourmilab.ch/cship/aberration.html
3: Walker, J. (2022). C-ship: The Doppler Shift. *wwww.fourmilab.ch*. Retrieved April 17, 2025, from https://www.fourmilab.ch/cship/doppler.html
4: Technologies, U. (2025). Unity - scripting API: Mesh.vertices. *unity3d.com*. Retrieved April 17, 2025, from https://docs.unity3d.com/ScriptReference/Mesh-vertices.html

Appendix

The Unity project, including C# scripts, shaders, Blender assets used for the London street scene, and potentially analysis scripts, are hosted on GitHub:

https://github.comLileafDayo/HUDRelativity.git